CSE312: Database Management Systems Lab

Lab 01: Entity–Relationship (ER) Diagram Using draw.io

1. Objective

The objectives of this experiment are:

- To understand the fundamental concepts of the Entity–Relationship (ER) model.
- To identify entities, attributes, and relationships in a real-world scenario.
- To construct an ER diagram using the draw.io diagramming tool.
- To represent weak entities, participation constraints, and various attribute types in an ER diagram.

2. Introduction

The Entity–Relationship (ER) model is a high-level conceptual data model used in the design of databases. It provides a graphical representation of the logical structure of a database, making it easier for designers and stakeholders to understand data requirements before implementation.

draw.io (also known as `diagrams.net`) is a free, browser-based diagramming tool that supports the creation of ER diagrams, flowcharts, UML diagrams, and more. It requires no installation and allows diagrams to be exported in various formats (PNG, SVG, PDF, XML).

Accessing draw.io:

- Open a web browser and navigate to `https://app.diagrams.net`.
- Choose a storage location (Device, Google Drive, GitHub, etc.) or select *Decide later*.
- Start a new blank diagram or choose the *Entity Relation* template from the template gallery.

3. Theory

3.1 Entity and Entity Type

Entity: An **entity** is a real-world object or concept that has an independent existence and can be distinctly identified. For example, a specific student *Alice* or the course *CSE301* are entities.

Entity Type: An **entity type** defines a collection (set) of entities that share the same attributes and properties. It represents the *schema* or blueprint for a group of similar entities. Example: `STUDENT`, `COURSE`, `EMPLOYEE`.

In an ER diagram, an entity type is represented by a **rectangle**.

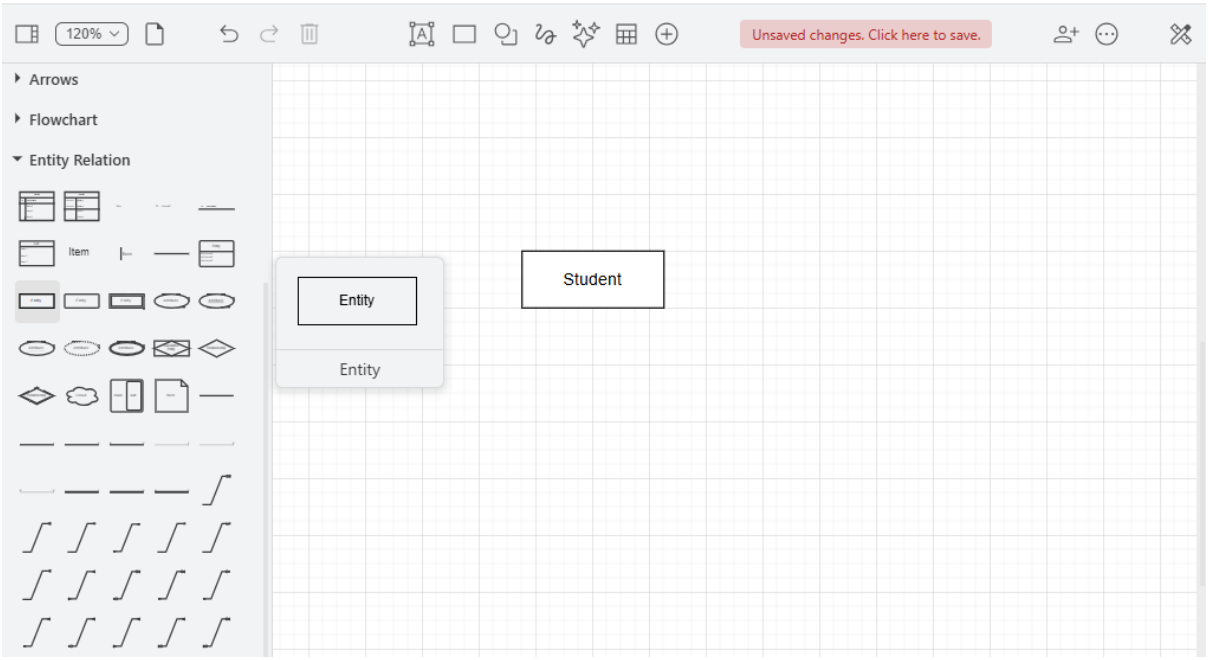

Figure 1: Entity type `STUDENT` represented as a rectangle in draw.io

3.2 Relationship and Relationship Type

Relationship: A **relationship** is an association among two or more entities. For example, the fact that student *Alice* is *enrolled in* course *CSE301* is a relationship instance.

Relationship Type: A **relationship type** defines a set of associations among entity types. It has a name and specifies which entity types participate in it. Example: `ENROLLS_IN` between `STUDENT` and `COURSE`.

In an ER diagram, a relationship type is represented by a **diamond** connected to the participating entity rectangles by lines.

The **degree** of a relationship is the number of participating entity types:

- **Binary** – two entity types (most common).
- **Ternary** – three entity types.
- **Unary (Recursive)** – one entity type related to itself.

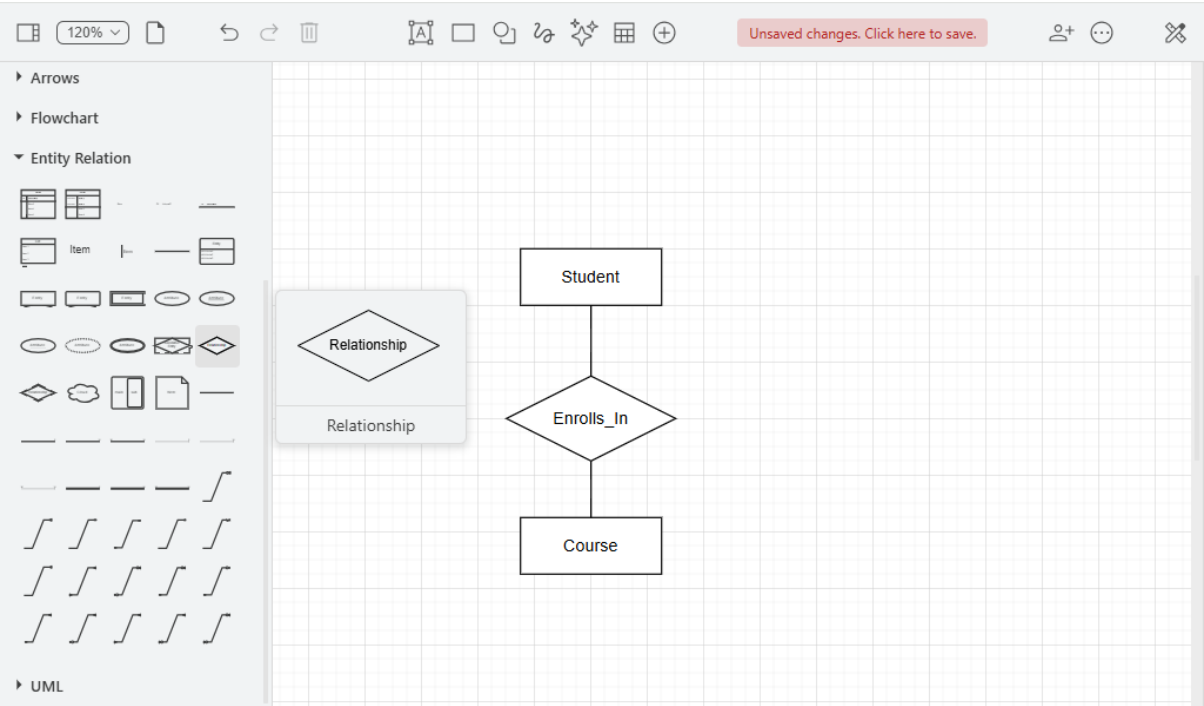

Figure 2: Relationship type `ENROLLS_IN` between `STUDENT` and `COURSE`

Cardinality of a relationship is the number of relationship instances each entity participates in. The cardinality of a relationship can be of the following types:

- **One-to-One (1:1)** – One entity of each side can form relationship with at most one other entity.
- **One-to-Many (1:M)** – Entities from *One* side can form relationship with more than one entities from *Many* side.
- **Many7-to-Many (M:M)** – Entities from both sides can form relationship with more than one entities from the other side.

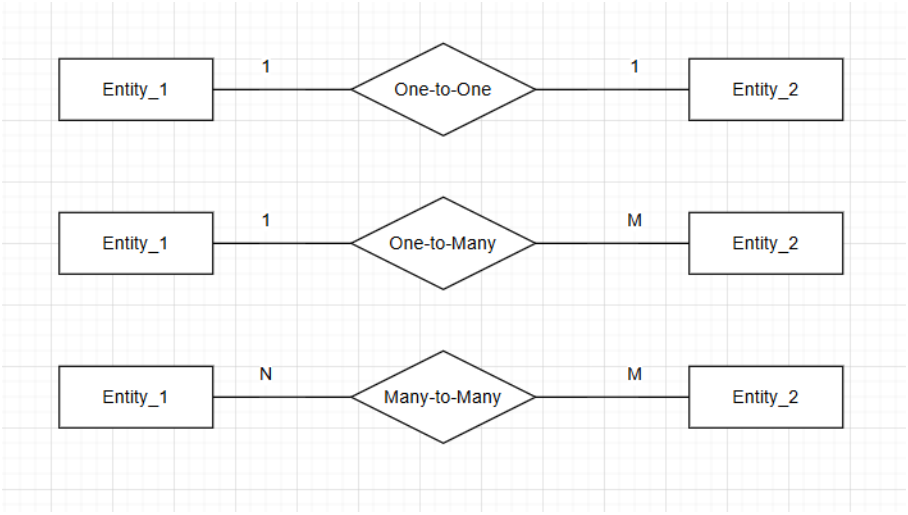

Figure 3: Different cardinalities in draw.io

3.3 Attributes and Their Types

Attribute: An **attribute** is a property or characteristic that describes an entity type or a relationship type. Example: `Name`, `Age`, and `StudentID` are attributes of `STUDENT`.

The following attribute types are commonly used in ER diagrams:

Attribute Type	Description	ER Notation
Simple (Atomic)	Cannot be divided further. e.g., `Age`, `Gender`	Oval
Composite	Can be divided into sub-parts. e.g., `Name` → First, Last	Oval with child ovals
Multi-valued	Can hold multiple values. e.g., `PhoneNumbers`	Double oval
Derived	Computed from another attribute. e.g., `Age` from `DOB`	Dashed oval
Key	Uniquely identifies each entity. e.g., `StudentID`	Oval with underlined text

Table 1: Types of attributes and their ER diagram notation

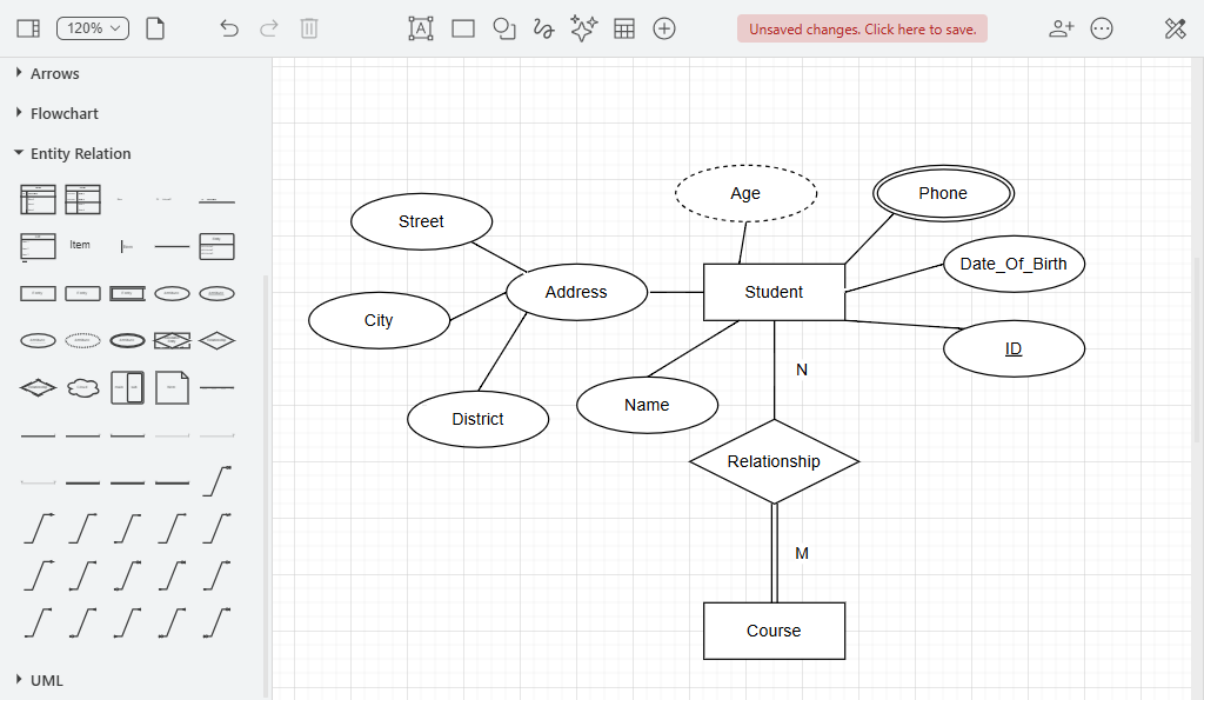

Figure 4: Different attribute types connected to the `STUDENT` entity in draw.io

3.4 Weak Entities

Weak Entity Type: A **weak entity type** is an entity type that does not have a key attribute of its own. It depends on another entity type (the **owner** or **strong** entity) for its identification. Example: `DEPENDENT` (of an employee) depends on `EMPLOYEE`.

Key characteristics of weak entities:

- Represented by a **double rectangle**.
- The relationship connecting it to its owner is called an **identifying relationship**, represented by a **double diamond**.
- A weak entity has a **partial key** (discriminator), underlined with a *dashed* line, that uniquely identifies it *within* the owner's context.
- A weak entity always has a **total participation** constraint with its identifying relationship.

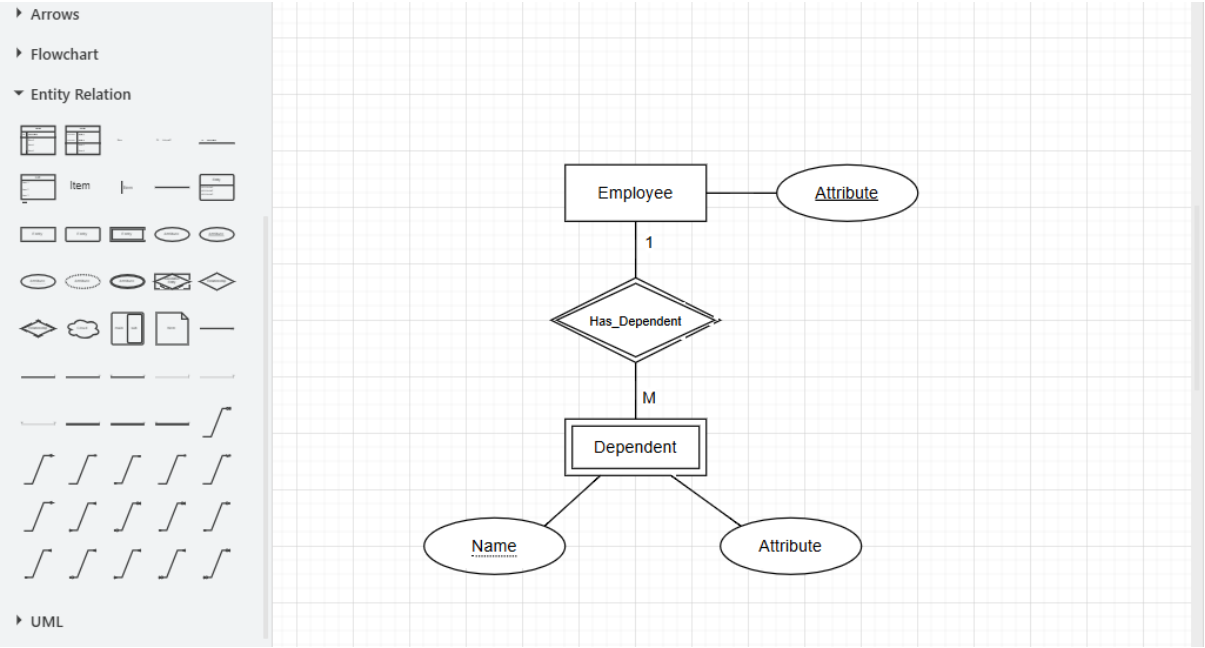

Figure 5: `DEPENDENT` as a weak entity of `EMPLOYEE` in draw.io

3.5 Participation Constraints

Participation Constraint: A **participation constraint** specifies whether the existence of an entity depends on it being related to another entity through a relationship.

There are two types:

- **Total Participation (Mandatory):** Every entity in the entity type *must* participate in at least one relationship instance. Denoted by a **double line** between the entity rectangle and the relationship diamond. *Example:* Every `EMPLOYEE` must work for a `DEPARTMENT`.
- **Partial Participation (Optional):** Some entities may not participate in any relationship instance. Denoted by a **single line**. *Example:* Not every `EMPLOYEE` manages a `DEPARTMENT`.

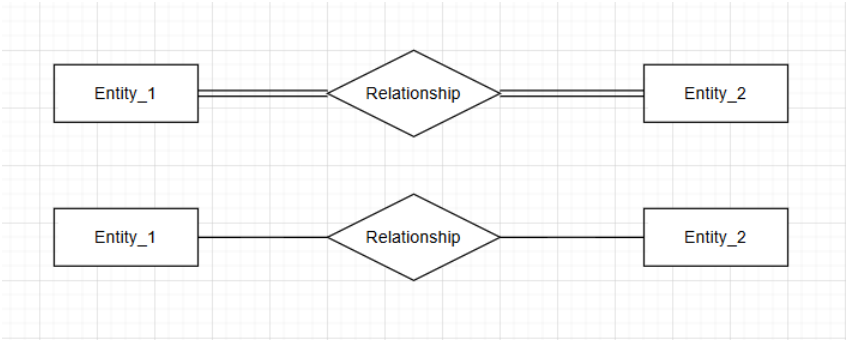

Figure 6: Total (double line) and partial (single line) participation in draw.io

4. Procedure

4.1 Opening draw.io and Selecting the ER Template

1. Open a browser and navigate to `https://app.diagrams.net`.
2. In the Shapes section, expand the **Entity Relation** template.

4.2 Adding Entities

1. From the left panel under *Entity Relation*, drag a **Entity** shape (rectangle) onto the canvas.
2. Double-click the shape to rename it (e.g., `STUDENT`).
3. For a **weak entity**, use the double-bordered rectangle shape.

4.3 Adding Attributes

1. Drag an **Attribute** shape (oval/ellipse) onto the canvas near the entity.
2. Double-click and name it (e.g., `StudentID`).
3. Connect it to the entity using the **connector** tool (hover over the entity border until a blue arrow appears, then drag to the attribute oval).
4. For a **key attribute**, underline the text: right-click the shape → *Edit Style* → add `fontStyle=4;` (value 4 = underline).
5. For a **multi-valued attribute**, use a double-bordered oval.
6. For a **derived attribute**, use a dashed-bordered oval: `dashed=1;`.

4.4 Adding Relationships

1. Drag a **Relationship** shape (diamond) onto the canvas between two entities.
2. Double-click and name it (e.g., `ENROLLS_IN`).
3. Connect the diamond to each participating entity using connectors.
4. Add **cardinality labels** (1, N, M) on the connector lines near each entity.
5. For an **identifying relationship** (weak entity), use a double-bordered diamond.

4.5 Setting Participation Constraints

1. Select the connector between an entity and a relationship diamond.
2. Right-click → *Edit Style*.
3. For **total participation**, change the line style to `strokeWidth=3;` or add a double-line by setting `endFill=1;` with a parallel line effect.
4. For **partial participation**, keep the default single-line connector.

4.6 Exporting the Diagram

1. Go to **File** → **Export As**.
2. Select **PNG** or **PDF** for submission.
3. Save the XML source via **File** → **Save** for future editing.

5. Observations

- Entity types were successfully represented using rectangles and named appropriately.
- Attributes of various types (simple, composite, multi-valued, derived, key) were attached to entities using the correct ER notation.
- Relationships were modelled using diamonds with cardinality labels (1:1, 1:N, M:N).
- Weak entities and their identifying relationships were distinguished using double-bordered shapes.
- Participation constraints were indicated using single and double connector lines.

6. Lab Task

Task A: University Management System (Guided)

Design an ER diagram for a **University Management System** with the following requirements:

1. **Entities:** `STUDENT`, `FACULTY`, `DEPARTMENT`, `COURSE`.
2. **Attributes:**
   - `STUDENT`: `StudentID` (key), `Name` (composite: First, Last), `PhoneNumbers` (multi-valued), `Age` (derived from DOB).
   - `COURSE`: `CourseCode` (key), `Title`, `Credits`.
   - `FACULTY`: `FacultyID` (key), `Name`, `Designation`.
   - `DEPARTMENT`: `DeptID` (key), `DeptName`, `Location`.
3. **Relationships:**
   - `STUDENT` *enrolls in* `COURSE` (M:N).
   - `FACULTY` *teaches* `COURSE` (1:N).
   - `DEPARTMENT` *offers* `COURSE` (1:N).
   - `FACULTY` *belongs to* `DEPARTMENT` (N:1, total participation for `FACULTY`).
4. Indicate all participation constraints appropriately.
5. Export the completed diagram as a PNG and attach it to your lab report.

Task B: Hospital Management System (Unguided)

Imagine a hospital management system where various elements need to be modeled.

In this system, there are components such as "Patient," "Doctor," and "Department." The "Patient" element has characteristics like "PatientID," "Name," "Address," and "DateOfBirth." The "Doctor" element includes details such as "DoctorID," "Name," "Specialization," and "YearsOfExperience." The "Department" component has identifiers like "DepartmentID," "DepartmentName," and "Location."

Consider the element "Appointment" that represents a scheduled interaction between a patient and a doctor. This component includes specifics like "AppointmentDate," "AppointmentTime," and "ConsultationFee." The "Appointment" element is closely connected to both the "Patient" and "Doctor" components and derives its existence from these connections.

Various interactions occur in this system. For example, one interaction might indicate that a "Patient" schedules an appointment with a "Doctor," where details like "Purpose-OfVisit" are recorded. Another interaction could show that a "Doctor" is associated with a "Department," indicating their work area.

Draw an Entity-Relationship Diagram based on the above description.

7. Conclusion

This experiment provided a hands-on introduction to the Entity–Relationship model and its practical application using the draw.io diagramming tool. Through identifying entities, attributes, and relationships, and representing them with the standard ER notation, students gained foundational skills required for conceptual database design. These skills form the basis for translating a real-world problem into a structured relational schema in subsequent experiments.